

CSA1711 - Assessment Tool 3

N. Prince Raj - 192312284

Descriptive Written Test

Intelligent Agents · Smart Home Automation · Uninformed Search in Maze Navigation

Question 1 — Intelligent Agents in a Smart Home Automation System

1.1 System Context

The smart home system must handle three continuous streams of real-time input — motion detection, temperature readings, and time-of-day — and act on three subsystems (lighting, temperature/HVAC, and security), while also learning individual user preferences over time. This combination of reactive control and long-term adaptation makes it a natural case study for comparing agent architectures.

1.2 Types of Intelligent Agents and Their Function in the System

(a) Simple Reflex Agent

A simple reflex agent selects actions using only the current percept, matched against a fixed set of condition–action rules (if–then), with no memory of the past and no model of the world.

- Function in this system: "IF motion detected in room THEN turn on light"; "IF temperature > 28°C THEN turn on AC"; "IF no motion for 10 minutes THEN turn off light."
- Limitation: it cannot distinguish why motion occurred (a resident walking through vs. a pet vs. an intruder), cannot factor in time-of-day context (e.g., it would light up a room at 3 a.m. exactly as it would at 3 p.m.), and cannot learn — every user gets identical treatment.

(b) Model-Based Reflex Agent

This agent maintains an internal state that tracks aspects of the world not visible in the current percept alone, updated as new percepts arrive, and applies condition–action rules to this internal state rather than to the raw percept.

- Function in this system: it can track "room occupied since 6:42 p.m.", "door was locked at last check", "outside temperature has been falling for the last hour", and combine this with the current percept — e.g., keep the light on because the room is still marked occupied even during a brief moment with no detected motion.
- Limitation: it still only reacts according to fixed rules over its state; it has no explicit notion of a goal to plan toward, and no way to compare or optimize between competing objectives (comfort vs. energy saving).

(c) Goal-Based Agent

A goal-based agent selects actions by considering future consequences and choosing actions that will achieve an explicitly represented goal, effectively doing a form of search or planning over possible action sequences.

- Function in this system: given the goal "room temperature = 22°C by 7:00 a.m. when the user wakes", the agent plans when to start the heater based on how long the room takes to warm, rather than reactively switching on/off at a fixed threshold. Similarly, given the goal "secure the house when everyone has left", it can sequence actions: lock doors → arm sensors → turn off non-essential devices.

- Limitation: goal-based agents are typically binary in their evaluation — a goal is either achieved or not — so they don't naturally express trade-offs like "slightly less comfortable but much more energy-efficient", which matters when comfort and efficiency goals conflict.

(d) Utility-Based Agent

A utility-based agent extends the goal-based agent with a utility function that scores how desirable each possible state or outcome is, allowing it to choose among multiple ways of achieving a goal — or among conflicting goals — by maximizing expected utility rather than simply satisfying a yes/no condition.

- Function in this system: it can weigh comfort against energy cost continuously — e.g., assign higher utility to "maintain 23°C using natural ventilation" than to "maintain 23°C using AC at peak electricity rate", even though both satisfy the same temperature goal. It can also balance security strictness against convenience, e.g., reducing sensitivity of motion-triggered alarms when a known family pet pattern is detected, to avoid false alarms while still weighting genuine intrusion risk highly.
- This is also the natural place to incorporate learning: a learning component can adjust the utility function itself over time (e.g., learning that this particular household values energy savings more than instant warmth), turning the utility-based agent into a learning agent.

1.3 Comparative Summary

Agent Type	Uses Memory?	Uses Goals?	Handles Trade-offs?	Learns Preferences?
Simple Reflex	No	No	No	No
Model-Based Reflex	Yes (internal state)	No	No	No
Goal-Based	Yes	Yes	No (binary)	Not inherently
Utility-Based	Yes	Yes	Yes	Yes, with a learning component

1.4 Justification: Most Effective Approach

No single classical agent type fully satisfies all three requirements — comfort, energy efficiency, and security — on its own, because each requirement demands a different strength:

- Security benefits from fast, low-latency reaction to certain unambiguous events (e.g., a door forced open at 2 a.m. should trigger an alarm instantly) — the strength of a simple reflex agent.
- Comfort and continuity of experience (e.g., not re-triggering lights every time someone briefly leaves a room) require the internal state tracking of a model-based agent.
- Coordinated, forward-looking behaviour, such as pre-heating a room before a scheduled wake time, requires the planning capability of a goal-based agent.
- Balancing energy cost against comfort, and adapting sensitivity/thresholds to the household's own patterns, requires the continuous trade-off evaluation and learning of a utility-based agent.

The most effective design is therefore a layered hybrid architecture, structured as:

- A simple-reflex safety layer for time-critical, unambiguous security events (fire/intrusion alarms), which always overrides other layers so critical response time is never delayed by planning or utility computation.
- A model-based layer that maintains room occupancy, device status, and short-term environmental trends, feeding accurate state into the layers above it.

- A goal-based planning layer that sequences actions toward explicit objectives (target temperature by a set time, "house secured" mode when unoccupied).
- A utility-based learning layer on top, which continuously tunes thresholds and weighs comfort against energy cost per household, personalizing behaviour over time and resolving conflicts between subsystems (e.g., lighting vs. HVAC energy budget).

This layered hybrid gives the system the instant reactivity needed for security, the contextual awareness needed for comfort continuity, the forward planning needed for efficiency, and the adaptive personalization that a purely reflexive or purely goal-based design cannot provide on its own.

Question 2 — Uninformed Search for Maze Navigation and Agent Design

2.1 Problem Framing

A robot placed in an unknown maze, with no prior map, must find a path to the exit. This is modelled as a search problem: states are the robot's possible positions (cells), actions are moves to adjacent open cells, the initial state is the robot's start cell, and the goal test checks whether the current cell is the exit. Because the robot has no heuristic knowledge of how far the exit is (it doesn't know the maze layout), only uninformed (blind) search strategies are directly applicable until enough of the maze has been discovered to justify heuristics.

2.2 Uniform Cost Search (UCS)

UCS expands the frontier node with the lowest cumulative path cost $g(n)$ so far, using a priority queue rather than a simple FIFO/LIFO queue. When all step costs are equal (as in a typical grid maze, 1 per move), UCS behaves identically to BFS; UCS becomes essential specifically when different moves might have different costs (e.g., diagonal moves, difficult terrain, or a "risky corridor" costing more).

- Advantage: guarantees the optimal (minimum-cost) path to the exit, even with non-uniform step costs, as long as all costs are non-negative.
- Advantage: complete — if a solution exists and the state space is finite (or costs are bounded below by a positive constant), it will be found.
- Disadvantage (time complexity): $O(b^{(1+LC^*/\epsilon)})$, where b is branching factor, C^* is optimal solution cost, and ϵ is the minimum step cost — in the worst case this can explore a very large number of maze cells before finding the exit, especially in a large or maze-like environment with many dead ends.
- Disadvantage (space complexity): $O(b^{(1+LC^*/\epsilon)})$ as well, since UCS must keep every generated frontier node in memory (as a priority queue) — this can be prohibitive for a large maze, which is a real constraint for an onboard robot with limited memory.

2.3 Iterative Deepening Search (IDS)

IDS repeatedly performs depth-limited depth-first search, increasing the depth limit by one each iteration (limit = 0, 1, 2, ...) until the goal is found. It combines depth-first search's low memory footprint with breadth-first search's completeness and optimality (for unit step costs).

- Advantage (space complexity): $O(b \cdot d)$, where d is the depth of the shallowest goal — dramatically lower memory than UCS/BFS, since it only needs to remember the current path (like DFS), not the entire frontier. This is highly attractive for a robot with limited onboard memory exploring a large, unknown maze.
- Advantage: complete and, for unit step costs (the common case in a grid maze), optimal — it is guaranteed to find the shortest path in terms of number of moves.

- Disadvantage (time complexity): $O(b^d)$ — nodes at shallow depths are regenerated on every iteration, so IDS re-does a lot of redundant work. In practice, this repeated shallow re-exploration is a relatively small overhead compared to the cost of a single exhaustive search at depth d , but it is still asymptotically the same order as plain DFS/BFS in the worst case.
- Disadvantage: if step costs are not uniform, IDS (in its basic form) does not guarantee the optimal-cost path — only UCS handles that case correctly.

2.4 Comparison Table

Criterion	Uniform Cost Search (UCS)	Iterative Deepening Search (IDS)
Expansion order	Lowest cumulative cost $g(n)$ first	Depth-first, up to an increasing depth limit
Time complexity	$O(b^{(1+LC^*/\epsilon)L})$	$O(b^d)$
Space complexity	$O(b^{(1+LC^*/\epsilon)L})$ — stores full frontier	$O(b \cdot d)$ — stores only current path
Complete?	Yes (non-negative costs)	Yes (finite depth)
Optimal?	Yes, for any non-negative step costs	Yes, only when step costs are uniform
Best suited when...	Move costs differ across the maze (risky zones, rough terrain)	Memory is limited and move costs are uniform

2.5 Relating This to Agent Types

The choice of search strategy interacts directly with the design of the agent carrying it out:

- A simple reflex agent cannot use UCS or IDS at all in any meaningful way, since both require constructing and reasoning over a search tree of possible future states — a simple reflex agent has no memory of the maze structure and cannot look ahead, so it could only wall-follow reactively (e.g., always turn right) with no optimality guarantee.
- A model-based reflex agent can incrementally build and remember a map of visited cells and known walls as it explores, which is what makes running UCS or IDS possible at all in an initially unknown maze — the agent's internal state is exactly the partial maze graph that the search algorithm operates over. Without this memory, the robot would repeatedly revisit the same cells with no record of where it had already been.
- A goal-based agent uses this internal map together with UCS/IDS as its planning mechanism: the explicit goal is "reach the exit cell", and the search algorithm is the mechanism used to find a sequence of actions from the current state to that goal.
- A utility-based agent becomes relevant if the maze has non-uniform costs (e.g., some corridors are longer, narrower, or riskier) and the robot must trade off path length against risk or energy consumption — here UCS's cost-minimizing behaviour is a direct instantiation of utility maximization, since "cost" is effectively the negative of utility.

Because the maze is unknown, this is also inherently an online search problem: the robot cannot compute a full plan before moving (it doesn't have the map yet), so in practice UCS or IDS is re-invoked repeatedly on the growing known-map as the robot explores and updates its internal state — mirroring the interleaved sense-plan-act cycle of a model-based, goal-driven online agent.

2.6 Recommended Combination and Justification

The most effective combination is a model-based, goal-based agent (i.e., an agent that maintains an internal map and pursues the explicit goal of reaching the exit), using Iterative Deepening Search as the default exploration/search strategy, with a switch to Uniform Cost Search once non-uniform costs are known to exist in the maze (e.g., after sensing a rough or risky corridor).

- Reasoning for the agent type: since the maze is initially unknown, the agent must remember what it has already explored (ruling out a simple reflex agent) and must be driven by the explicit goal of reaching the exit rather than by fixed condition–action rules alone (ruling out a pure model-based reflex agent) — this points to a model-based, goal-based hybrid.
- Reasoning for IDS as the default: a physical robot has limited onboard memory, and a maze is exactly the kind of environment where the search tree can be deep and highly branching, so IDS's $O(b \cdot d)$ space complexity is a decisive practical advantage over UCS's exponential frontier storage, while still guaranteeing the shortest path when all moves cost the same (the common case for a physical robot moving one cell at a time).
- Reasoning for switching to UCS when needed: if the maze introduces variable-cost terrain (e.g., a corridor requiring slower, more careful — hence more "expensive" — movement, or a zone the utility function marks as risky), only UCS guarantees the minimum-cost solution in that setting, so the agent's search module should be cost-aware and fall back to UCS whenever cost non-uniformity is detected.

In summary, the maze-exploration problem is best solved by an agent that is model-based (to remember the maze as it is discovered) and goal-based (to be explicitly driven toward the exit), applying IDS by default for its superior memory efficiency, and upgrading to UCS specifically when the environment is discovered to have non-uniform movement costs — giving the robot the best available balance of memory efficiency, completeness, and optimality guarantees appropriate to what it currently knows about the maze.